

AI Capability Vision & Discovery — What We Want From the AI, and Why

Purpose: Define — from a pure **AI / agentic point of view** — what intelligence this product should have, *why each capability earns its place*, and the open questions to confirm with stakeholders before we build or update.

How to use this doc:

- Each capability has: **What we want** → **Why (the value & the AI reason)** → **Example cases** → **Confirm (the question to pin it down)**.
- This is an AI-product spec disguised as a discovery doc. Read it to align on *what the AI is for*; use the Confirm lines to close gaps with the facility/eng.
- Record decisions inline as > **Decision:** and convert them into [FUTURE_TASKS.md](#) items or ADRs.

Where we are today (the AI baseline we're extending):

- On-prem Ollama (qwen2.5:14b reasoning · llama3.1:8b tools/SQL · llama3.2 auditor), open-source-only, zero data egress.
- 5 agent modes (investigator, optimizer, brief, root_cause, maintenance) on a custom ReAct loop, 6 read-only tools, 8-step cap.
- pgvector RAG over HVAC markdown · NL→SQL query · vision endpoint · streaming answers.
- Anti-hallucination stack (T1/T2/T3 guardrails + numeric/equipment/citation/language postchecks) · 27/27 deterministic eval.
- **Read-only by design** — the AI advises, it never controls the plant.

See [AGENT_REFERENCE.md](#) for the current agent/tool detail and [AI_FRAMEWORK_MIGRATION.md](#) for the open-source mandate.

Status legend (used per capability below):

-  **Have it** — built and working today
-  **Partial** — exists but incomplete / not wired everywhere
-  **Want it** — net-new, not built yet

The questions to ask — at a glance

Every capability below ends with a **Confirm:** line — that *is* the question for that topic. Here they all are in one place so you can walk the session top-to-bottom.

Each question is tagged by what it helps you capture:

-  **Purpose** — *why* we're building this / the highest-value job the AI must do
-  **Requirement** — a must-have the build has to satisfy
-  **Enhancement** — an upgrade to scope/prioritize, not a blocker

Start with the 🎯 Purpose questions (P1–P4) — they decide what "good" even means. (Full rationale + example cases are in the matching section.)

#	Topic	Helps capture	The question to ask
P1	Highest-value job	🎯 Purpose	If the AI could do only <i>one</i> thing well, what is it? (cut energy cost / catch faults early / speed up root-cause / shift handover)
P2	Real operator questions	🎯 Purpose	What are the top 5 questions operators actually ask about the plant today? (these become the modes, tools & eval cases)
P3	Cost of a wrong answer	🎯 Purpose	What's the worst outcome if the AI is confidently wrong? (sets how hard the truthfulness gate must be)
P4	AI persona & language	🎯 Purpose	What tone (terse vs explanatory) and what language(s)? (English-only is enforced today — is that final?)
0	AI thesis	🎯 Purpose	Is "ask anything + proactively surface issues, advisory only " right — or do they want the AI to <i>act</i> ?
1	Language understanding	📐 Requirement	How non-technical is the <i>least</i> technical user? (sets how forgiving the NL layer must be)
2	Reasoning style	📐 Requirement	Free ReAct (LLM picks tool order) vs guided playbooks ("if anomaly → root_cause path") — which do operators trust?
3	Autonomy ceiling ★	🎯 Purpose	What's the ceiling — A advisory / B soft-actions-with-approval / C operator-applied / D closed-loop? If B+, which actions and who approves?
4	New tools	⚡ Enhancement	Rank the candidate tools (cost/tariff, weather, maintenance-history, forecast). Which data behind them actually exists?
5	Truthfulness	📐 Requirement	Is "every claim cited + numbers provably correct" a hard gate or best-effort? (decides S2/S3 build)
6	Explainability	📐 Requirement	How much reasoning trace do operators want to see vs find noisy?
7	Knowledge / RAG	⚡ Enhancement	What documents must the AI cite, in what formats, and who keeps them correct?
8	Memory	⚡ Enhancement	How long should the AI remember? Is cross-session equipment memory worth prioritizing?
9	Proactive AI	⚡ Enhancement	Is proactive output wanted, and where should it land (in-app / Slack / email)?
10	Vision	⚡ Enhancement	Real operator need or demo nicety?

#	Topic	Helps capture	The question to ask
11	Models / hardware	 Requirement	Is open-source-only firm with no exceptions? What GPU + concurrency are committed?
12	Evaluation	 Requirement	What's the pass bar for "trustable" — green eval + demo / expert sign-off / provable numbers?
13	Feedback loop	 Enhancement	Do we commit to collecting  /  now so the learning loop + future fine-tune have data?
14	Delivery	 Enhancement	Which surfaces do operators and management actually want?
15	Reliability / latency	 Requirement	What answer latency is acceptable, and what's the degraded-mode expectation?

P. Purpose discovery — the "why are we building this" questions

Answer these *first*. They define what "good" means; every requirement and enhancement below is downstream of them.

P1 — The highest-value job

What we want to learn: The single AI capability that justifies the project on its own. **Why:** Focuses the whole build. If it's "catch faults early," proactive intelligence (§9) jumps to the top; if it's "speed up root-cause," the investigator agent (§2) does. **Example cases:** energy-cost reduction · early fault/degradation detection · faster root-cause · faster shift handover · sales/demo credibility.

Confirm: If the AI could do only one thing well, which is it? Everything else is secondary.

P2 — The real operator questions

What we want to learn: The actual questions operators ask the plant today, in their words. **Why:** These become the agent modes, the tools we prioritize (§4), and the golden eval cases (§12). Building modes nobody asks is wasted effort. **Example cases:** "why is chiller 2 using more power?" · "which chiller should run as lead?" · "is anything about to fail?" · "what did the night shift miss?"

Confirm: What are the top 5 questions operators ask? Capture them verbatim.

P3 — The cost of a wrong answer

What we want to learn: What happens if the AI is confidently wrong. **Why:** Directly sets how strict the truthfulness gate (§5) and eval bar (§12) must be. A wrong number that triggers a needless service call is costlier than a vague answer. **Example cases:** operator acts on a bad recommendation · trust collapses and adoption dies · a real fault is missed because the AI said "fine."

Confirm: What's the worst realistic outcome of a confident hallucination here?

P4 — AI persona & language

What we want to learn: How the AI should *sound*, and in what language(s). **Why:** Tone and language are prompt-level requirements that shape every answer. English-only is enforced today (with a language postcheck) — confirm that's final, not a temporary POC choice. **Example cases:** terse one-liners for floor operators vs explanatory paragraphs for engineers · English-only · English + a local language for some operators.

Confirm: What tone, and is English-only the final requirement or do we need multilingual later?

👤 **The ask-ready session script** (every question with cases + **Answer:** lines, no rationale) now lives in its own file: [AI_DISCOVERY_QUESTIONNAIRE.md](#) — print/share that for the meeting. This doc keeps the *why* + *current build status* behind each question (\$0–\$15 below).

0. The one-sentence AI thesis

What we want: A plant operator should be able to ask the system *anything* about the chiller plant in plain English and get a grounded, explainable, action-oriented answer — and the system should also tell them things they didn't think to ask.

Why: The data already exists in MySQL; the gap is that turning it into a decision takes a trained engineer and time. The AI's job is to collapse "raw telemetry → insight → action" from hours to seconds, *without ever fabricating a number*.

Confirm: Is "ask anything + proactively surface issues, advisory only" the right thesis — or does the facility actually want the AI to *act* (see §3)?

1. Conversational understanding — "ask anything in plain English"

Status: 🟡 Partial — NL Q&A + NL→SQL work today; typo/fuzzy matching is the open gap (T1-D).

What we want: Natural-language Q&A over live plant data — typos, shorthand, and operator jargon all understood. ("chillor 1 kwtr last nite", "why's CT-1 acting up").

Why:

- Operators won't learn query syntax. The AI is only adopted if asking is as easy as talking.
- It's the lowest-friction entry point and the most demo-able capability.
- AI reason: this is where LLMs are genuinely better than dashboards — intent parsing + synthesis.

Example cases:

- *Case A — Typo tolerance:* "chillor 1" must resolve to **chiller_1** (fuzzy match — currently a top backlog gap, T1-D).
- *Case B — Vague intent:* "is the plant ok?" → AI picks **brief** mode automatically.
- *Case C — Multi-clause:* "compare chiller 1 and 2 efficiency and tell me which to run as lead" → decompose into compare + recommend.
- *Case D — Free-text → SQL:* "show me kW for chiller 2 between 2 and 4am" → safe NL→SQL (column allow-list enforced).

Confirm: How non-technical is the *least* technical user? That sets how forgiving the language layer must be.

2. Agentic reasoning — "investigate, don't just answer"

Status: 🌀 Have it — 5 modes + orchestrator on a custom ReAct loop, all passing eval.

What we want: An agent that chains tools to *reach a conclusion* — fetch data, compute efficiency, detect anomalies, correlate, then explain the root cause — not a single-shot chatbot.

Why:

- Real operator questions ("why did efficiency drop after 3pm?") require multi-step investigation, not a lookup.
- This is the core differentiator vs a BI dashboard: the AI does the analytical legwork an engineer would.
- AI reason: ReAct tool-calling is what turns an LLM from "talks about data" into "reasons over data."

Example cases:

- *Case A — Investigator:* "why is chiller 2 underperforming?" → equipment list → timeseries → efficiency → anomalies → root-cause narrative.
- *Case B — Optimizer:* "how do we cut cost this week?" → compare chillers → loss drivers → tower approach → ranked actions.
- *Case C — Orchestration:* a planner fans out investigator + optimizer + maintenance in parallel, then synthesizes one report.

Confirm: Free exploration (current — LLM picks tool order) vs **guided playbooks** ("if anomaly → root_cause path"). Guided = more predictable/auditable but rigid (this is the LangGraph trigger). Which do operators trust more?

3. Autonomy ceiling — "advise vs act" ★ (the defining AI decision)

Status: 🌀 Have it (Case A, advisory) — `propose_work_order` exists (🌀) but no action is ever taken without being built out + approved.

What we want: Clarity on how far the agent's autonomy goes. Today it is strictly advisory.

Why: This single choice defines the entire safety model, the tool set, and liability. Principle #6 today: *the platform never controls the plant*. Any move beyond advisory reopens everything.

Example cases (pick the ceiling):

- *Case A — Pure advisory (current):* reads telemetry, writes prose. No writes anywhere. Safest.
- *Case B — Soft actions in OUR systems:* agent *drafts* a work order / maintenance ticket / Slack alert → **a human approves** before anything is created. (`propose_work_order` tool exists; the approve-UI doesn't yet.)
- *Case C — Operator-applied suggestions:* AI recommends a setpoint change; an operator manually applies it in the BMS. AI never writes to the BMS.

- *Case D — Closed-loop control:* AI writes setpoints directly. **We consider this out of scope / unsafe — confirm it stays out.**

Confirm: What is the ceiling — A, B, C, or D? If B+, *which* actions and *who* approves? This is the first question to answer; everything else bends to it.

4. The agent's toolbox — "what the AI is allowed to look at and do"

Status: 🟡 Partial — 6 read-only analytics tools today; the candidate tools below are 🟢 net-new.

What we want: The right set of tools so the agent can answer real questions, not just the 6 read-only analytics tools it has today.

Why:

- The agent is only as smart as its tools — each new tool unlocks a class of questions.
- AI reason: well-scoped, typed, validated tools are how we keep an LLM grounded and safe.

Example cases — candidate tools we likely want:

- *Cost/tariff tool:* kWh + time-of-use rate → ₹ impact. Turns "save energy" into "save ₹X/week." (Needs tariff data — do we have it?)
- *Weather/ambient tool:* outdoor wet-bulb to explain cooling-tower approach temp. (Explains causes the data alone can't.)
- *Maintenance-history tool:* "when was chiller 2 last serviced?" → makes **maintenance** mode real, not generic. (Needs a CMMS/log source.)
- *Forecast tool:* "what will load be at 3pm?" — the forecaster exists as a page; expose it to the agent.
- *Production-schedule tool:* explains load swings the AI currently can't account for.
- *Report-writer tool:* generates the weekly management summary.

Confirm: Rank these. Which one, if added, changes an operator's behavior tomorrow? Which data sources behind them actually exist (see §7)?

5. Grounding & truthfulness — "never make up a number"

Status: 🟡 Have it — T1/T2/T3 guardrails + numeric/equipment/citation/language postchecks + premise verification. S2/S3 deeper checks are 🟢.

What we want: Every numeric claim traceable to a computed value; every recommendation tied to real data; refusal when the data doesn't support an answer.

Why:

- One fabricated kW/TR figure destroys operator trust permanently — and in a plant, acting on a wrong number has real cost.
- This is *the* hardest and most important AI property here. It's why we built the T1/T2/T3 guardrails + postchecks.
- AI reason: LLMs hallucinate numbers fluently; we treat every LLM byte as untrusted until a code-side check validates it.

Example cases:

- *Case A — Numbers from analytics, not arithmetic*: the LLM cites kW/TR from the SUMMARY block; it never computes it itself.
- *Case B — Premise verification*: "fix the 2am spike" → if the data shows no spike, the AI refuses rather than inventing a fix (T2-I, already shipped).
- *Case C — Numeric reference check (S3, planned)*: recompute the answer in Python; flag any cited number >5% off ground truth.
- *Case D — Citation enforcement*: operational claims carry [source: ...] markers tied to retrieved chunks.

Confirm: Is "every operational claim must cite, numbers provably correct" a *hard* acceptance gate, or best-effort? This decides whether we build S2 (LLM-as-judge) and S3 (numeric compare).

6. Explainability & trust — "show your work"

Status: 🕒 Partial — live reasoning trace + analyzer audit panel exist; agent-UI audit panel is the open gap.

What we want: The operator can always see *how* the AI reached an answer — the tool trace, the numbers used, and any audit flags.

Why:

- Engineers won't act on a black box. Visible reasoning is what converts a skeptical operator into a user.
- AI reason: an exposed reasoning trace is also our best debugging and eval signal.

Example cases:

- *Case A — Live reasoning trace*: the agent UI streams "calling compute_efficiency → got 0.71 → calling detect_anomalies..." (exists on /agent).
- *Case B — Audit panel on every answer*: numeric/equipment/citation/language flags shown below the answer (exists on analyzer; **not yet on agent UI** — a top backlog gap).
- *Case C — Confidence signaling*: root_cause mode states a confidence level rather than implying certainty.

Confirm: How much trace does the operator *want* to see vs find noisy? (Engineers want it all; floor operators may want it hidden by default.)

7. Knowledge & RAG — "know HVAC, not just our data"

Status: 🕒 Partial — pgvector RAG + citations over curated markdown; PDF/Word/wiki ingest is 🕒.

What we want: The AI grounded in HVAC domain knowledge (playbooks, equipment manuals, O&M procedures) so its advice is expert-level, not generic LLM guesswork.

Why:

- Telemetry tells *what* is happening; the knowledge base tells *what it means and what to do*.
- AI reason: RAG is how we inject facility-specific, citable expertise without fine-tuning.

Example cases:

- *Case A — Curated markdown (current)*: our HVAC playbooks in pgvector.
- *Case B — OEM manuals (PDF) / O&M procedures (Word)*: triggers LangChain loader migration (Stage 4).
- *Case C — Facility wiki (SharePoint/Confluence)*: needs a connector + auth.

Confirm: What documents must the AI cite, in what formats, and who keeps them correct when an operator spots a wrong procedure?

8. Memory & context — "remember the conversation and the plant's history"

Status: 🟡 Partial — session memory via Postgres Thread/Message tables; cross-session equipment/preference memory is 🟢.

What we want: The AI holds conversation context within a session, and ideally recalls prior findings about a piece of equipment.

Why:

- Follow-ups ("and chiller 2?", "what about yesterday?") only work with memory.
- Recalling "we flagged this bearing last week" makes the AI feel like a colleague, not a stateless tool.
- AI reason: threaded memory (Postgres Thread/Message tables exist) is cheap context that sharply improves perceived intelligence.

Example cases:

- *Case A — Session memory (have it)*: multi-turn within one analyzer/agent session.
- *Case B — Equipment memory*: "last 3 anomalies you flagged on chiller 1" persists across sessions.
- *Case C — Operator preference memory*: remembers a user prefers ₹ over kWh, terse over verbose.

Confirm: How long should the AI "remember," and does cross-session equipment memory add enough value to prioritize?

9. Proactive intelligence — "tell me before I ask"

Status: 🟡 Want it — anomaly scan exists, but scheduled briefs / alerts / trend-watch digests are net-new.

What we want: The AI surfaces problems on its own — a morning brief, an anomaly alert, a "chiller 2 efficiency degrading 3 days straight" nudge — not only answering when asked.

Why:

- The biggest operational wins come from catching drift *early*; an operator can't ask about a problem they don't know exists.
- AI reason: this shifts the product from reactive Q&A to an always-on analyst — a major value step.

Example cases:

- *Case A — Scheduled morning brief*: auto-generated plant status at shift start (brief mode on a cron).
- *Case B — Anomaly-triggered alert*: z-score scan fires → AI writes a plain-English explanation + suggested check.

- *Case C — Trend watch*: "kW/TR up 8% over 3 days" digest, even with no hard anomaly.

Confirm: Is proactive output wanted, and where should it land — in-app, Slack, or email (see §13)? This is a meaningful build, so confirm appetite before scoping.

10. Multi-modal — "look at a photo or a chart"

Status: 🟡 Partial — a vision endpoint exists; specific use cases (nameplate/gauge/chart) are unscoped.

What we want: The AI interprets images — a gauge photo, a nameplate, a screenshot of a trend — alongside text.

Why:

- Operators photograph things on the floor; letting them ask "what's wrong here?" with a picture is high-value and demo-friendly.
- AI reason: a vision endpoint already exists; the question is how much to invest.

Example cases:

- *Case A — Nameplate read*: photo → extract design capacity into the catalog.
- *Case B — Gauge/HMI read*: photo of a panel → AI reads the value and comments.
- *Case C — Chart critique*: paste a trend screenshot → AI explains the shape.

Confirm: Is vision a real operator need or a demo nicety? That decides priority.

11. Model strategy & quality — "right brain for the right job, all open-source"

Status: 🟡 Have it — 3-model right-sizing (ADR locked), open-source-only, zero egress. vLLM/fine-tune are 🟡 trigger-gated.

What we want: Each task served by the right-sized open-weights model; quality high enough that operators trust answers; everything on-prem.

Why:

- A 14B model for SQL generation is wasteful; a 3B for root-cause reasoning is too weak. Right-sizing controls latency and VRAM.
- Open-source-only protects data sovereignty (telemetry never leaves the building) — currently a locked architectural mandate.
- AI reason: per-task model selection is a documented ADR; the open ceiling is the main quality risk.

Example cases:

- *Case A — Current right-sizing*: qwen2.5:14b reasoning · llama3.1:8b tools/SQL · llama3.2 auditor.
- *Case B — Scale-up*: vLLM swap when >10 concurrent operators or <5s SLA (Stage 1 migration).
- *Case C — Domain fine-tune*: LoRA on 6+ months of operator-labeled Q&A for HVAC-specialist quality.

Confirm: Is the open-source-only mandate firm with *no* exceptions (even Azure OpenAI Private Endpoint)? And what GPU + concurrency are committed — that decides Ollama vs vLLM and model size.

12. Evaluation & quality assurance — "prove the AI is right, repeatedly"

Status:  Partial — 27/27 deterministic eval + smoke test; S2 (LLM-as-judge) and S3 (numeric reference) are .

What we want: A regression-proof way to know the AI's quality before every release — and to catch quality drift.

Why:

- Prompt or model changes silently break answers; without eval we ship regressions blind.
- AI reason: a golden eval suite is the only thing that makes LLM changes safe to deploy.

Example cases:

- *Case A — Deterministic eval (have it):* 27 golden cases, refusal cases, equipment/outlier/language pins.
- *Case B — Semantic eval (S2, planned):* a second local model judges whether each claim is grounded in context.
- *Case C — Numeric reference (S3, planned):* Python ground-truth vs cited numbers, >5% flagged.
- *Case D — Live feedback → eval:* operator  /  turns failed answers into new eval cases.

Confirm: What's the pass bar before a build is "trustable" — green eval + demo (current), or expert sign-off on N real answers, or provable numeric correctness?

13. Learning loop — "get better from operator feedback"

Status:  Want it — the `analysis_audit` table can hold a verdict, but the  /  UI + review queue are net-new.

What we want: A  /  (and optional comment) on every AI answer that feeds a review queue and, eventually, model improvement.

Why:

- It's the cheapest signal for what's actually wrong, and it builds the labeled dataset a future fine-tune needs.
- AI reason: closes the loop from "we think it's good" to "operators confirm it's good."

Example cases:

- *Case A — Verdict capture:*  /  → `analysis_audit.operator_verdict` → weekly review.
- *Case B — Failed answers become eval cases:* every  is triaged into the golden suite.
- *Case C — Fine-tune fuel:* 6 months of verdicts → LoRA on llama3.1:8b.

Confirm: Do we commit to collecting feedback now (small build) so the learning loop and future fine-tune have data later?

14. Delivery of intelligence — "meet operators where they are"

Status:  Partial — web app only today; Slack bot + scheduled email/PDF are  (Phase 9A planned).

What we want: AI insight delivered through the right surface — web app, Slack/Teams, scheduled email/PDF — not only a page someone has to open.

Why:

- An insight nobody sees has zero value; delivery surface is part of the AI product, not an afterthought.
- AI reason: it pairs directly with \$9 proactive intelligence — proactive output needs a channel.

Example cases:

- *Case A — Web app only (current).*
- *Case B — Slack morning brief / alerts (planned Phase 9A).*
- *Case C — Weekly management PDF by email (report-writer mode + scheduler).*

Confirm: Which surfaces do operators and management actually want? This sizes the proactive/reporting work.

15. Reliability of the AI experience — "graceful when the brain is offline"

Status:  Have it — circuit breaker, degraded-mode banners, graceful refusals, Postgres startup retry.

What we want: When Ollama/Tailscale/DB is down or slow, the AI degrades with a clear message — never a bare 500, never a fabricated answer from stale data.

Why:

- Trust is lost as fast at failure time as at hallucination time. "I can't reach the model right now" beats a wrong answer.
- AI reason: we already have circuit breakers + degraded banners; the principle is "fail loudly, refuse gracefully."

Example cases:

- *Case A — Model down:* circuit breaker opens → degraded banner, analyzer disabled cleanly.
- *Case B — Slow under load:* streaming keeps the wait tolerable; or trigger vLLM if the SLA tightens.
- *Case C — Stale data:* answer states "as of HH:MM" rather than implying live.

Confirm: What answer latency is acceptable (streaming at ~30s OK, or hard <10s?), and what's the degraded-mode expectation?

Quick decision summary (fill in during the session)

#	AI capability decision	Decision	→ Task / ADR
0	AI thesis (advise vs act)		

#	AI capability decision	Decision	→ Task / ADR
1	Language forgiveness level (least-technical user)		
2	Reasoning style: free ReAct vs guided playbooks		
3	Autonomy ceiling (A/B/C/D)		
4	Top new tools to add (+ data behind them)		
5	Truthfulness gate: hard or best-effort?		
6	How much reasoning trace to show operators		
7	RAG document formats + KB owner		
8	Memory horizon (session / equipment / preference)		
9	Proactive intelligence wanted? where?		
10	Vision: real need or demo nicety		
11	OSS mandate firm? GPU + concurrency		
12	Eval pass bar for "trustable"		
13	Commit to feedback loop now?		
14	Delivery surfaces (web/Slack/email)		
15	Latency SLA + degraded-mode expectation		

Last updated: 2026-06-02 · This is the AI-capability spec; pair with [FUTURE_TASKS.md](#) to turn decisions into work and [AI_FRAMEWORK_MIGRATION.md](#) for the open-source constraint.